HIRESENSE AI

AI-Powered Resume Intelligence & Candidate Evaluation System

PROJECT DOCUMENTATION

Document information	Details
Project	HireSense AI
Document version	1.0
Status	Working MVP; deployment and planned enhancements remain future work
Prepared by	Sangeetha S
Documentation date	5 November 2026
Repository	github.com/sangeethakumar-dev/hire-sense-ai-resume-intelligence

Purpose: document the implemented MVP, architecture, technical decisions and rationale, workflow, testing considerations, limitations, and proposed roadmap. Future features are labelled as planned rather than completed.

Document Guide

- 1. Executive summary and objectives
- 2. Problem statement and scope
- 3. Implemented features
- 4. Architecture and data flow
- 5. Technology stack and design decisions
- 6. Data models, matching and scoring
- 7. Caching and reliability
- 8. User interface
- 9. Repository structure and setup
- 10. Testing and limitations
- 11. Security and responsible AI
- 12. Deployment and future roadmap
- 13. Submission checklist and glossary

1. Executive Summary

HireSense AI is a recruitment decision-support MVP that helps a recruiter compare multiple PDF resumes against a supplied job description. It extracts structured candidate profiles, analyses job requirements, performs keyword and semantic matching, identifies skill gaps, calculates matching indicators, ranks candidates, and presents individual candidate reports in a Streamlit interface.

The implementation combines Google Gemini for structured language-model analysis, Sentence Transformers for local text embeddings, ChromaDB for persistent vector storage, Pydantic for structured validation, and Python services for orchestration and matching.

The MVP was run locally with one job description and multiple candidate resumes. A successful observed run displayed three evaluated candidates, candidate rankings, candidate selection, matching breakdowns, skills analysis, education and experience information, and candidate reports. The source changes were pushed to the GitHub main branch.

This document does not claim production deployment, authentication, multi-tenant operation, or a complete audit/compliance framework.

2. Problem Statement, Scope and Objectives

2.1 Problem

Manual resume screening requires recruiters to read documents, extract details, compare qualifications with job criteria, and prepare shortlists. This can be repetitive, time-consuming and difficult to standardise.

2.2 Objective

Provide a recruiter-facing tool that organises resume information and offers consistent comparison signals to support human review.

2.3 MVP objectives

- Accept one job description and multiple PDF resumes.
- Extract candidate information into structured profiles.
- Extract job title, skills, education and experience criteria from a job description.

- Compare profiles using keyword and semantic signals.
- Identify matched required skills, missing required skills and matched preferred skills.
- Evaluate education eligibility when the required information is available.
- Rank evaluated candidates and present individual reports.
- Reuse cached resume data and vectors for identical resumes.
- Retry selected temporary Gemini service failures with bounded backoff.

2.4 Not included in the current MVP

- Public production deployment and production-grade operations.
- User login, registration, password reset and role-based access control.
- Organisation workspaces, tenant isolation and team collaboration.
- Autonomous hiring decisions or automatic candidate rejection.
- Completed legal compliance or fairness certification.

3. Implemented Features

Feature	Current behaviour
Job description upload	Recruiter uploads a supported text-based job description through Streamlit.
Multiple resume upload	Recruiter uploads multiple PDF resumes in one analysis session.
Resume extraction and parsing	PDF text is extracted and converted into a structured candidate profile.
JD analysis	Gemini analyses the job description and returns a structured job profile.
Embeddings	Sentence Transformers model all-MiniLM-L6-v2 generates local embeddings.
Vector persistence	ChromaDB stores resume chunks and metadata for retrieval and reuse.
Candidate matching	Matching logic calculates keyword, semantic, skill and experience indicators.
Education eligibility	Degree and explicitly configured graduation-year criteria are evaluated separately.
Candidate ranking	Evaluated candidates are ranked and shown in a table.
Candidate reports	The dashboard displays profile, score breakdown, skills, education, experience and projects.
Caching	File hashing identifies identical resume files so cached profile/vector data can be reused.
Retry handling	Temporary Gemini failures such as selected 503/429/server errors are retried a bounded number of times.

4. System Architecture

The MVP uses a modular Python backend with a Streamlit presentation layer. The recruitment pipeline coordinates the main processing stages.

Recruiter → Streamlit upload interface → Recruitment pipeline → Resume processing and JD processing → Structured profiles → Embedding/vector storage → Candidate matching → Ranking/report generation → Dashboard

4.1 Main modules

Module / file	Responsibility
app.py	Streamlit setup, uploads, pipeline invocation, summary metrics, ranking and candidate report display.
backend/agents/resume_parser.py	Gemini-based structured parsing of resume text.
backend/agents/job_analyser.py	Gemini-based extraction of job requirements.
backend/agents/candidate_matcher.py	Candidate/job comparison, matching indicators and eligibility logic.
backend/models/schemas.py	Pydantic schemas for resume, job, match, evaluation and ranking data.
backend/services/resume_extractor.py	Extracts text from resume documents.
backend/services/resume_processor.py	Validates, hashes, parses, chunks and stores resume data; checks cache first.
backend/services/text_chunker.py	Splits extracted text into chunks.
backend/services/vector_store.py	Manages ChromaDB, file hashes, vector storage, retrieval and cached metadata.
backend/services/jd_processor.py	Processes job description text and calls the JD analyser.
backend/services/recruitment_pipeline.py	Coordinates JD processing, resumes, matching, ranking and report assembly.
backend/services/candidate_ranking.py	Ranks candidate evaluations.
backend/services/candidate_report.py	Creates structured candidate report content.

4.2 End-to-end workflow

1. Recruiter uploads a job description and multiple PDF resumes.
2. The app stores uploads temporarily for the duration of pipeline execution.
3. The JD processor extracts text and calls the Gemini analyser.
4. Each resume is validated and hashed; the processor checks ChromaDB for an existing record.
5. For a new resume, text is extracted, parsed, chunked, embedded and stored.
6. For a cached resume, the existing structured profile and vectors are reused where available; legacy records may require a one-time parse and metadata update.
7. The matching engine compares each candidate profile with the job profile.
8. The pipeline collects successful evaluations, records errors, ranks candidates and prepares reports.
9. Streamlit renders metrics, candidate rankings and individual candidate details.

5. Technology Stack and Design Decisions

Technology	Role
Python	Primary language for agents, services, matching and orchestration.
Streamlit	Rapid recruiter-facing MVP interface.
Google Gemini API	Natural-language analysis and structured extraction.
LangChain / LangChain Google GenAI	LLM integration and structured prompt/model workflow.
Sentence Transformers	Local embeddings using all-MiniLM-L6-v2.

Technology	Role
ChromaDB	Persistent local vector store for resume chunks and metadata.
PyMuPDF	PDF text extraction.
Pydantic	Typed schemas and validation of structured outputs.
Pandas	Tabular data handling where used.
Git / GitHub	Version control and source hosting.

5.1 Streamlit decision

Streamlit was selected because it supports file upload controls, metrics, tables, selection controls and report sections in one Python application. This enabled a working recruiter workflow without first building a separate JavaScript frontend and API client.

5.2 Gemini decision

Gemini handles language-understanding tasks such as extracting structured information from resumes and interpreting free-form job descriptions. Structured Pydantic outputs make the results easier for downstream Python services to validate.

5.3 Local embeddings decision

all-MiniLM-L6-v2 generates embeddings locally. This separates embedding generation from Gemini analysis and avoids depending on a separate paid embedding API call for every resume chunk.

5.4 ChromaDB decision

ChromaDB provides persistent vector storage and retrieval suitable for the local MVP. Production use would require further design for managed storage, access control, backups and tenant isolation.

5.5 Resume caching decision

A resume may be evaluated against multiple job descriptions. Re-parsing and re-embedding the same file would waste time and compute/API resources. File hashing identifies identical files so cached profiles and vectors can be reused while candidate matching runs afresh for each new JD.

5.6 Retry decision

Gemini may temporarily return 503 UNAVAILABLE, 429 RESOURCE_EXHAUSTED or selected server errors. Bounded retries with increasing wait intervals improve resilience to transient failures. They do not guarantee success during a persistent outage.

5.7 Education eligibility decision

Graduation-year eligibility was added as a separate job-profile field because degree-only checks can incorrectly pass a candidate when the JD restricts eligible graduation years. The intended outcomes distinguish eligible, ineligible and not verified. Missing data should not silently count as a pass.

5.8 Separate match score from eligibility

A candidate may have strong skill similarity but fail a hard eligibility requirement. The dashboard therefore displays matching scores and education eligibility separately. A high matching score is not proof of eligibility or a hiring recommendation.

6. Data Models, Matching and Scoring

6.1 ResumeProfile

The profile contains full_name, email, phone, location, professional_summary, technical_skills, soft_skills, education, work_experience, projects and certifications. Education includes institution, degree, field_of_study and graduation_year. Work experience includes company, job_title, start_date, end_date and descriptions.

6.2 JobDescriptionProfile

The job profile contains job_title, required_skills, preferred_skills, minimum_experience, education, graduation_year_eligibility and responsibilities.

6.3 CandidateMatchResult

The match result includes overall_match_score, keyword_match_score, semantic_match_score, required_skill_score, preferred_skill_score, experience_score, education_eligible, matched_skills, missing_required_skills, matched_preferred_skills, matching_evidence, missing_requirements and match_summary.

6.4 Matching signals

- **Keyword match:** overlap between job requirements and candidate information.
- **Semantic match:** embedding-based estimate of meaning-level relevance.
- **Required skill score:** matching against required skills.
- **Preferred skill score:** matching against preferred skills.
- **Experience score:** output of the configured experience-matching logic.
- **Education eligibility:** separate eligibility indicator.

The exact scoring weights and formula should be read from the current candidate_matcher.py source. This document does not invent or freeze numeric weights.

Eligibility result	Meaning
True / Eligible	Available education information satisfies the configured degree and year criteria.
False / Not eligible	Available information shows that the candidate does not satisfy the configured criteria.
None / Not verified	Information is absent or insufficient to make a reliable determination.

Eligibility depends on extraction quality. Recruiters should verify dates and qualifications against the original documents.

7. Resume Caching, Vector Storage and Reliability

7.1 Resume identity

The vector-store service calculates a file hash to identify an identical resume file. If the file content changes, its hash may change and it can be treated as a new document.

7.2 New resume processing

1. Validate file existence and supported file type.
2. Calculate file hash and check for an existing record.
3. Extract resume text.
4. Parse text into a structured candidate profile.
5. Chunk text and generate embeddings.
6. Store chunks and metadata in ChromaDB, including candidate identifiers and cached profile data where configured.
7. Return the candidate profile and identifier for matching.

7.3 Cached resume processing

When a matching file hash is found, the processor retrieves cached profile and vector metadata. If a structured profile exists, it can be reused without repeating parsing and embedding generation. The candidate is still matched against the newly supplied JD.

7.4 Legacy cache records

Older vector records may not contain serialized profile data. The compatibility path can parse such a resume once, update existing vector metadata with the profile and reuse the existing vectors rather than regenerating embeddings.

7.5 Retry behaviour

Resume parsing and JD analysis were updated to retry selected temporary provider errors with bounded exponential backoff. The implemented example uses up to three attempts with waits of two and four seconds between attempts. Non-retryable errors are surfaced without repeated attempts.

8. User Interface and Recruiter Experience

The current Streamlit interface uses a light background with purple accents, a central HireSense AI heading, two upload areas, a prominent Analyse Candidates button and a report section.

- Upload one job description and multiple PDF resumes.
- Show upload confirmation messages.
- Prominent Analyse Candidates button.
- Summary metrics for submitted resumes, evaluated candidates, reused duplicates and processing errors.
- Candidate ranking table and candidate selector.
- Candidate overview and matching score breakdown.
- Compact colour-coded chips for matched, missing and preferred skills.
- Expandable sections for technical skills, education, work experience, projects and matching evidence.
- Downloadable candidate report text when available.

The upload cards were changed to native Streamlit components after an earlier custom HTML approach displayed raw HTML in the interface. Limited HTML/CSS remains for presentation details such as skill chips and button styling.

9. Repository Structure

Path	Purpose
app.py	Streamlit application entry point.
backend/agents/	Resume parsing, JD analysis and candidate matching.
backend/models/schemas.py	Pydantic data models.
backend/services/	Extraction, processing, vector storage, pipeline, ranking and report services.
data/job_descriptions/	Sample job-description files, subject to privacy and repository decisions.
tests/	Unit and pipeline-oriented test files.
list_models.py	Gemini model-listing utility used during development.
requirements.txt	Python package dependencies.
.env.example	Environment configuration template; placeholders only.

Path	Purpose
.gitignore	Excludes secrets, private resumes, local databases, generated outputs and local environments.

Keep this inventory aligned with the actual repository as files evolve.

10. Local Setup and Execution

10.1 Prerequisites

- Python compatible with the installed project dependencies.
- Git.
- A Google Gemini API key with access to the configured model.
- An environment capable of running Streamlit and the Python dependencies.

10.2 Clone and install

Run the following commands in a terminal:

```
git clone https://github.com/sangeethakumar-dev/hire-sense-ai-resume-intelligence.git
cd hire-sense-ai-resume-intelligence
python -m venv .venv
.venv\\Scripts\\activate
pip install -r requirements.txt
```

10.3 Configure environment

Create a local .env file based on .env.example and provide the Gemini API key using the exact environment-variable name expected by the application. Do not commit .env or real credentials.

10.4 Run the application

```
streamlit run app.py
```

Streamlit normally opens at http://localhost:8501. The app uses a local ChromaDB persistence directory. Local vector databases should not be committed.

11. Testing, Validation and Known Limitations

11.1 Development validation performed

- The application was run locally with one JD and three candidate PDF resumes.
- A successful observed run displayed three evaluated candidates and their rankings.
- The individual candidate selector was checked for all three candidates.
- Candidate profile sections, score breakdown and skill analysis were visually checked.
- Gemini 503 failures were observed during development; retry handling was added to resume and JD analysis.
- Education eligibility was updated to consider graduation-year restrictions; the observed 2018 graduate was marked not eligible for a 2025/2026-only JD.
- Git changes were committed and pushed to the GitHub main branch.

11.2 Known limitations

- LLM extraction can be incomplete or incorrect; extracted profiles and job criteria need human verification.
- Matching scores are algorithmic indicators and need calibration against representative labelled examples.

- Experience scoring should be tested for fresher roles, minimum experience, internships, career gaps and date overlap.
- Graduation-year extraction should be tested against varied wording, ranges, missing years and conflicting criteria.
- A successful run alone does not prove every cache path was exercised; cached reuse should be verified with tests or instrumentation.
- Local ChromaDB storage is not a production multi-user architecture.
- The current MVP does not include authentication, organisation-level access control or cloud deployment.
- No production security, privacy, fairness or legal certification is claimed.

11.3 Recommended test cases

Area	Suggested tests
Resume parsing	Valid PDF, scanned PDF, malformed PDF, missing contact information, multiple education records.
JD extraction	Required/preferred skills, no education criteria, explicit year restriction, ambiguous criteria.
Matching	Exact skill, synonym, semantic relevance, missing required skill, no preferred skills.
Eligibility	Accepted degree/year, wrong year, missing year, unsupported degree, missing education.
Caching	New resume, identical re-upload, same resume/new JD, legacy record without cached profile.
Reliability	Gemini 503/429 then success, persistent failure, non-retryable validation error.
UI	No uploads, one resume, multiple resumes, partial processing error, empty results.

12. Security, Privacy and Responsible AI

- Resumes contain personal information. Store them only where access is controlled and retain them only as long as needed.
- Do not commit resumes, private datasets, API keys, .env files, credentials or local vector databases.
- Use .gitignore to exclude private data and generated local artifacts.
- Restrict access to any deployed application and its database.
- Treat scores as decision-support information, not automatic hiring decisions.
- Provide human review for extracted information, eligibility and rankings.
- Before real recruitment use, assess applicable privacy, employment, data-protection and anti-discrimination requirements.

The local MVP is not represented as a certified or legally compliant hiring system.

13. Deployment Plan and Future Roadmap

The following are proposed future enhancements. They are not implemented in the current MVP.

13.1 Planned feature 1 — User authentication and access control

- Secure registration and login.
- Password hashing and secure session/token handling.
- Password reset and account lifecycle workflows.
- Role-based access for recruiter and administrator roles.
- Authorisation so users can access only permitted candidate records and reports.

- Audit events for important account and candidate-data actions.

13.2 Planned feature 2 — Recruiter workspace and organisation management

- Organisation/workspace creation and membership.
- Associate job descriptions, candidate pools and reports with a workspace.
- Recruiter invitations and role/permission management.
- Saved job descriptions, shortlists and candidate review status.
- Tenant isolation across database queries, vector retrieval and document storage.

13.3 Planned feature 3 — Cloud deployment and production infrastructure

- Deploy the application to a suitable cloud platform with HTTPS.
- Evaluate managed or securely hosted vector storage to replace local-only ChromaDB persistence.
- Use managed secrets rather than source-code credentials.
- Use durable object storage with encryption, access controls and retention policies.
- Add monitoring, structured logs, error tracking, backups and health checks.
- Configure CI/CD for automated tests and controlled releases.
- Load-test document processing, embedding generation and concurrent sessions.

13.4 Additional enhancements

- Candidate search and filtering by skills, experience, education and match indicators.
- Support more document formats and scanned-PDF OCR.
- Recruiter feedback and configurable job-specific scoring weights.
- Evidence-based explainability for each match.
- Fairness and bias evaluation with legally reviewed procedures.
- Asynchronous processing and job queues for large batches.
- PDF export of formatted candidate reports.
- Administrative usage and system-health dashboard.

13.5 Suggested phases

Phase	Focus	Exit criteria
1 — MVP	Current local Streamlit workflow	Core upload, parse, match, rank and report flow works locally.
2 — Security	Authentication, roles and workspace model	Protected data and verified access-control tests.
3 — Cloud readiness	Cloud storage, managed database, secrets and monitoring	Staging deployment with backups and secure configuration.
4 — Production	CI/CD, load tests and operational controls	Production readiness review and monitored release.
5 — Enhancements	Search, feedback, explainability and more formats	Validated improvements based on recruiter feedback.

14. Submission and Maintenance Checklist

- README.md explains the project, stack, setup and run commands.

- requirements.txt includes the dependencies required by the application.
- .env.example contains placeholder configuration only.
- .gitignore excludes local databases, private resumes, secrets and generated test files.
- No personal resumes, API keys or credentials are committed.
- The app can be installed and launched from a clean clone.
- A non-sensitive sample JD is included only if appropriate.
- Tests are run and results recorded before release.
- README and this document are updated when architecture or setup changes.
- Future capabilities are labelled as planned, not completed.

15. Glossary

Term	Meaning
LLM	Large Language Model, used for natural-language extraction and analysis.
Embedding	Numerical vector representation of text used for semantic comparison.
ChromaDB	Vector database used to persist and retrieve resume chunk embeddings.
Semantic matching	Similarity estimation based on vector representations.
Keyword matching	Comparison based on explicit terms and skills.
Pydantic	Python library used to define and validate structured data models.
Cache	Previously processed data retained for reuse.
MVP	Minimum Viable Product, an initial working version validating core functionality.
RBAC	Role-Based Access Control; permissions assigned according to a user's role.
CI/CD	Continuous Integration and Continuous Delivery/Deployment.

16. Project Ownership

Project: HireSense AI
Author: Sangeetha S
Repository: https://github.com/sangeethakumar-dev/hire-sense-ai-resume-intelligence
Current delivery: Local working MVP; cloud deployment and authentication are planned future enhancements.

End of document.